



OS-Quest: Progress Report

Phase 2: Implementation & Development Tracking

Team 506 | April 2026

Team Members:

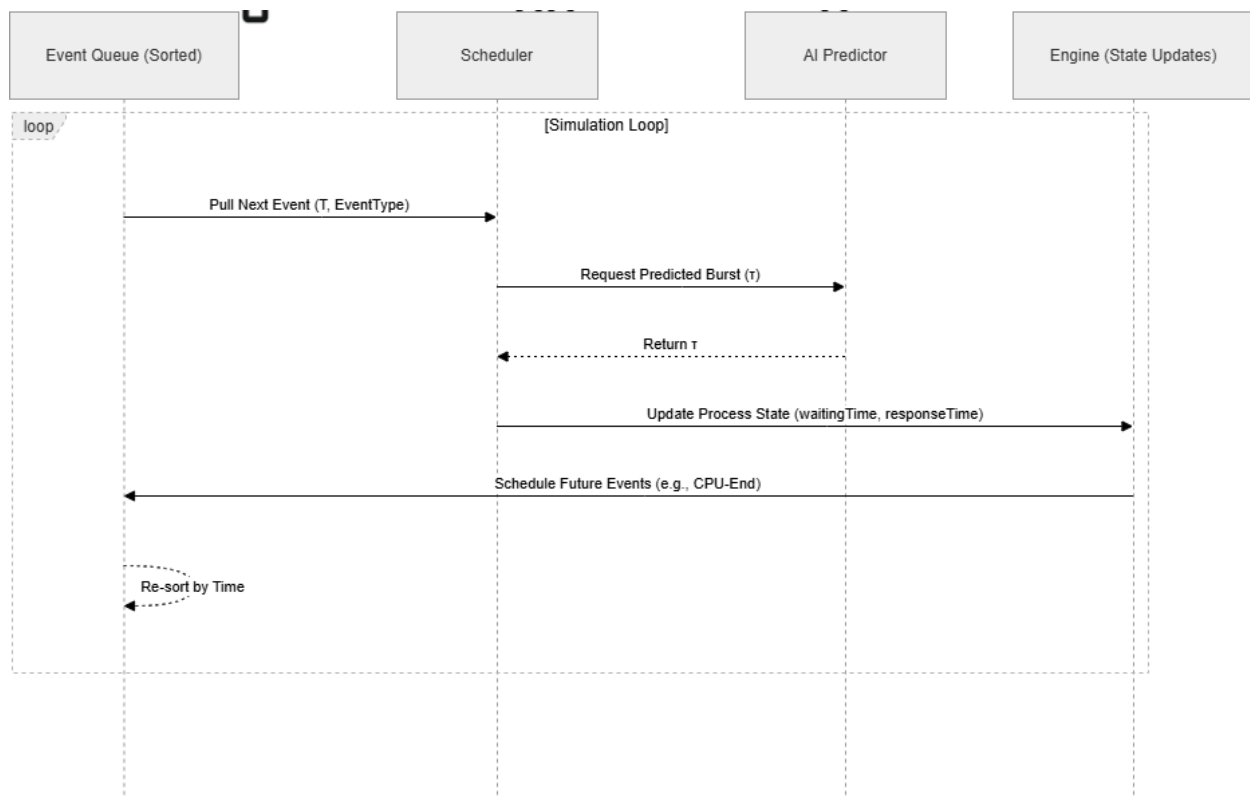
Faisal Alabbas

Abdulrahman Ammar

Riyadh Alkhudhayri

1. Project Recap

OS-Quest is an interactive, high-performance CPU scheduling simulator designed to bridge the gap between theoretical Operating System concepts and modern Artificial Intelligence. Moving beyond basic static calculations, this project utilizes a formal Discrete Event-Driven Engine and Machine Learning to simulate realistic system behavior. The project features a unique "Human vs. AI" mode where users compete against an AI-optimized scheduler to manage system resources effectively.



2. Current Implementation Status

Significant progress has been made in transitioning the project from a high-level design to a functional system.

- 1- **Event-Driven Core (C):** The core engine is now fully operational in C, managing a formal timeline of discrete events: Arrival, CPU-Burst End, I/O-Start, I/O-End, and Preemption.
- 2- **AI Prediction Logic:** The Exponential Smoothing formula:

$$\text{Tau}_{\{n+1\}} = \alpha * t_n + (1 - \alpha) * \text{Tau}_n.$$

$\text{Tau}_{\{n+1\}}$ represents the predicted value for the next CPU burst.

t_n is the actual length of the most recent CPU burst.

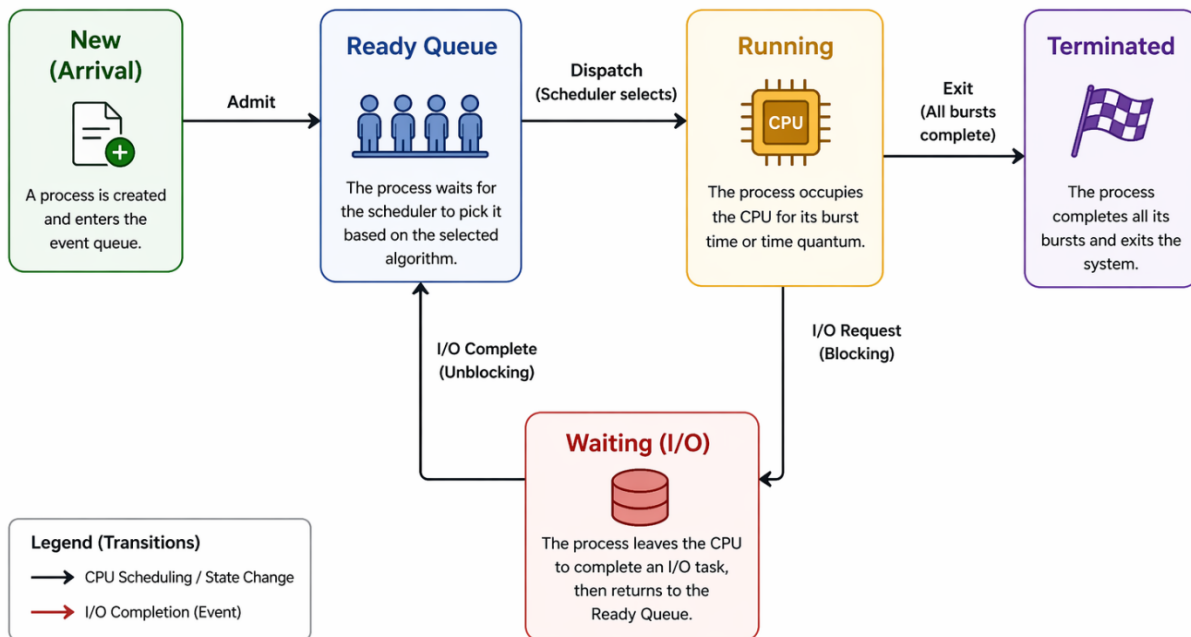
Tau_n is the previous prediction made by the system.

α is a smoothing factor (typically between 0 and 1) that determines how much weight is given to recent history versus older history.

The formula has been integrated into the Shortest-Job-First (SJF) module to predict future burst times based on history.

- 3- **Web Dashboard:** The primary interface has moved to a "Cyberpunk-Professional" React dashboard featuring dark mode and glass morphism.
- 4- **Scheduling Policies:** FCFS and SJF are fully implemented; Round Robin and Priority algorithms are currently in the final testing phase.

Process State Transition Diagram



3. System Architecture

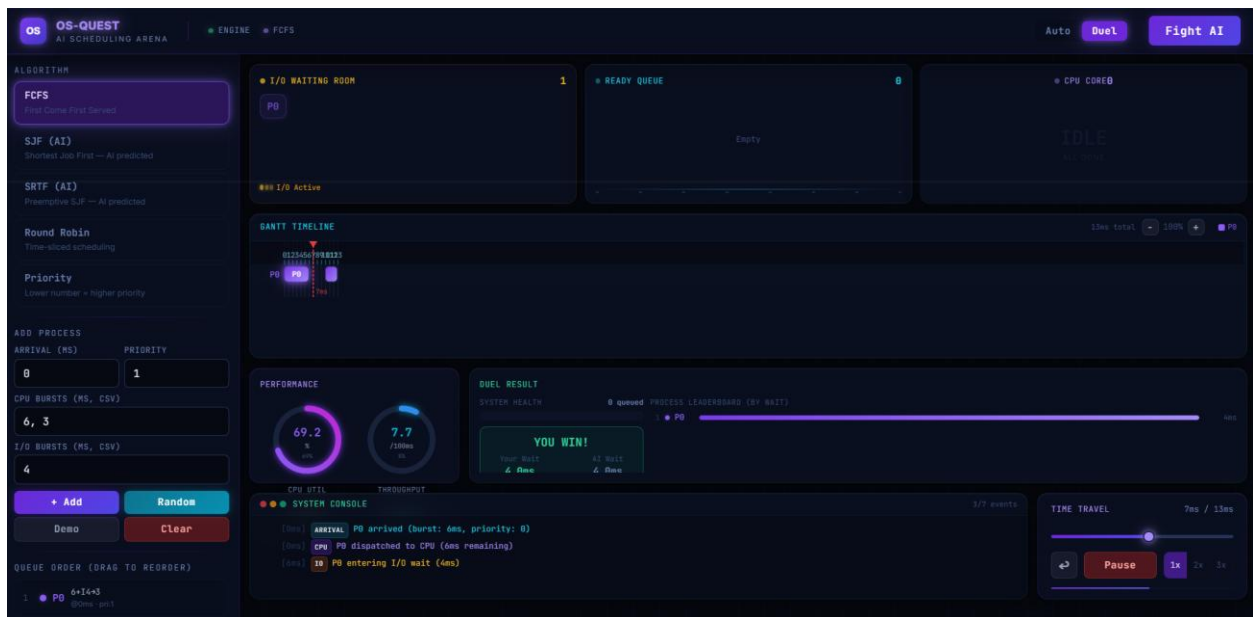
The architecture ensures modularity and near native performance.

Layer	Responsibility	Technology
Interface Layer	Handles process entry, algorithm selection, and gamification controls.	React, Tailwind CSS
Logic Engine	Runs the event queue and scheduling algorithms.	C (via WebAssembly)
AI Predictor	Calculating predicted next bursts using historical data.	C Logic
Visualizer	Renders the Gantt chart and real-time performance gauges.	Framer Motion, JS

4. Screenshots of Working System & Code



Auto Mode Dashboard Screenshot #1



Dual Mode Dashboard Screenshot #2

```
File Edit Selection View Go Run Terminal Help
c_engine 2 c_scheduler X
1 #include "scheduler.h"
2 #include <string.h>
3 #include <stdlib.h>
4
5
6 /* Internal helpers
7  */
8
9 /* Min-heap event queue */
10 static Event eq[MAX_EVENTS];
11 static int eq_size = 0;
12
13 static void eq_push(Event e) {
14     int i = eq_size++;
15     eq[i] = e;
16     /* bubble up */
17     while (i > 0) {
18         int parent = (i - 1) / 2;
19         if (eq[parent].time > eq[i].time ||
20             (eq[parent].time == eq[i].time && eq[parent].type > eq[i].type)) {
21             Event tmp = eq[parent]; eq[parent] = eq[i]; eq[i] = tmp;
22             i = parent;
23         } else break;
24     }
25 }
26
27 static Event eq_pop(void) {
28     Event top = eq[0];
29     eq[0] = eq[--eq_size];
30     /* bubble down */
31     int l = 0;
32     while (1) {
33         int r = 2*l+1, r = 2*l+2, smallest = l;
34         if (l < eq_size && (eq[l].time < eq[smallest].time ||
35             (eq[l].time == eq[smallest].time && eq[l].type < eq[smallest].type)))
36             smallest = l;
37         if (r < eq_size && (eq[r].time < eq[smallest].time ||
38             (eq[r].time == eq[smallest].time && eq[r].type < eq[smallest].type)))
39             smallest = r;
40         if (smallest == l) break;
41         Event tmp = eq[smallest]; eq[smallest] = eq[l]; eq[l] = tmp;
42         l = smallest;
43     }
44     return top;
45 }
46
47 /* Ready queue (simple array, re-sorted each time we pick next process) */
48 static int ready[MAX_PROCESSES];
```

The Logic in C code

5. Challenges Faced

The transition from a basic web simulator to a high-depth, event-driven system introduced several technical hurdles:

- 1- **Asynchronous State Management:** Integrating the C-based logic engine (compiled via WebAssembly with the React frontend proved challenging. Ensuring that the "Gantt Chart" renders smoothly while the C engine processes high-frequency events requires precise synchronization to avoid UI freezing.
- 2- **I/O Burst Complexity:** Implementing the Waiting (I/O) state required a fundamental rewrite of the scheduling logic. Managing processes that leave the CPU for I/O and return to the Ready Queue at a future time, without disrupting the event timeline, added significant complexity to the engine.
- 3- **AI Predictor Tuning:** Calibrating the Exponential Smoothing formula to accurately predict the next burst was difficult. We had to experiment with different alpha values to ensure the "Smart SJF" actually outperformed standard algorithms in noisy workloads.
- 4- **Event Queue Integrity:** Maintaining a perfectly sorted Event Queue in C is critical. Any error in event ordering (e.g., a process arriving at the exact same time a burst ends) can lead to simulation deadlocks or incorrect metric calculations.

6. Plan for Completing Remaining Work

With the core engine and AI logic established, our focus moves toward final algorithmic implementation and gamification features:

Phase 2 Implementation (In Progress)

- 1- **Finalize Round Robin & Priority Logic:** Complete the integration of time-quantum preemption and priority-based queue reordering in the C engine.
- 2- **Metric Engine Validation:** Run extensive tests to ensure that Throughput, CPU Utilization, and Waiting Times are calculated with 100% accuracy according to standard OS formulas.

Gamification & Expo Features

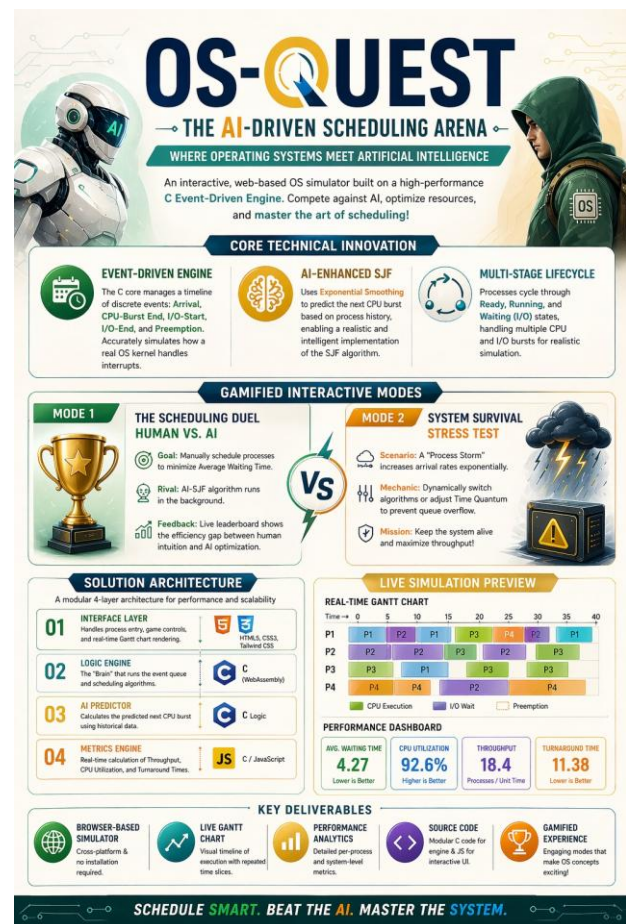
- 1- **The "Dual" Mode Interface:** Build the interactive drag-and-drop UI that allows expo visitors to manually schedule processes against the AI.
- 2- **System Survival Mode:** Implement the "Process Storm" logic where arrival rates increase over time to test the system's stability and the user's quick-thinking skills.

Final Polish

- 3- UI/UX Refinements: Apply the "Cyberpunk" aesthetic across all components, including glowing gauges and smooth Framer Motion transitions for process state changes.
- 4- Final Documentation: Compile the comprehensive Phase 2 report, including screenshots of the working system and final comparative analysis of the AI vs. Standard algorithms.

7- Conclusion

In conclusion, OS-Quest transforms a traditional CPU scheduling simulator into a high-performance, event-driven system that mirrors real-world operating system kernel logic. By transitioning from static loops to a discrete event-driven engine in C, the project now supports complex, multi-burst process lifecycles including realistic I/O wait states. The integration of AI-driven burst prediction, using the exponential smoothing formula mentioned which provides a technical "edge" by solving the classic scheduling problem of predicting the future behavior of incoming jobs. Beyond technical depth, the addition of gamified expo modes, such as the "Scheduling Duel" and "System Survival," ensures that the project is not only an academic success but also an engaging tool for public demonstration.



Our System Expo Poster